
1. Konzeptionsphase Portfolio

Objektorientierte und funktionale Programmierung mit Python

DLBDSOOFPP01_D

Dennis Simon Lätsch (IU14096102)

Softwareentwicklung (B.SC.)

26. Mai 2026

Tutor: Dr. Aiko Walte

Abbildungsverzeichnis

1	Wireframe des geplanten Studien-Dashboards	2
2	UML-Klassendiagramm der Entity-Klassen	4
3	CSV-Datei schreiben	6
4	GUI-Machbarkeitstest mit Tkinter	6
5	Konsolenausgabe des CSV- und Prognose-Machbarkeitstests	7

Tabellenverzeichnis

1 Metriken des Studien-Dashboards 1

1 Zielsetzung des Studien-Dashboards

Ziel des Projekts ist die Erstellung eines Studien-Dashboards zur Überwachung meines Studienfortschritts im Studiengang B.Sc. Softwareentwicklung. Das Dashboard soll zentrale Kennzahlen übersichtlich darstellen und frühzeitig sichtbar machen, ob die persönlichen Studienziele realistisch erreichbar sind. Es soll dabei als begleitendes Werkzeug für das flexible Fernstudium an der IU dienen.

Im Mittelpunkt stehen Kennzahlen und Metriken, die durch das Dashboard fortlaufend überwacht werden können. Dazu gehören ein angestrebter Notendurchschnitt, eine transparente Darstellung des aktuellen ECTS-Fortschritts sowie die aktuelle Bearbeitungsgeschwindigkeit, die als Velocity bezeichnet wird. Aus dieser Velocity soll eine Prognose des voraussichtlichen Studienendes berechnet werden.

Eine zentrale Grundlage für diese Berechnungen ist die Modulübersicht. Sie stellt alle Module tabellarisch dar und enthält Informationen zu Status, Name, Prüfungsform, ECTS und Note. Gleichzeitig soll sie als zentraler Ort für die spätere Interaktion mit dem Dashboard dienen. So sollen Module hinzugefügt werden können, beispielsweise Wahlmodule, die erst im Studienverlauf gewählt werden. Ebenso sollen Noten eingetragen und der Bearbeitungsstatus eines Moduls, etwa auf *in Arbeit* oder *fertig*, geändert werden können.

Tab. 1. Metriken des Studien-Dashboards

Metrik	Erklärung und Ziel
Allgemeine Studieninformationen	Studienstart, Ende der Regelstudienzeit und Studiengangsbezeichnung.
ECTS-Verteilung	Die ECTS werden nach den Statuswerten anerkannt, fertig, in Arbeit und offen gruppiert. Zusätzlich soll ein Prozentwert der abgeschlossenen und anerkannten ECTS im Verhältnis zu den noch offenen ECTS dargestellt werden.
Notenschnitt	Der aktuelle gewichtete Notendurchschnitt wird angezeigt und mit dem angestrebten Zielwert verglichen.
Velocity	Die Velocity beschreibt die durchschnittlich erreichten ECTS pro Monat. Sie wird aus den abgeschlossenen ECTS und der bisherigen Studiendauer berechnet.
Prognose	Aus den noch offenen ECTS und der aktuellen Velocity wird eine Prognose für das voraussichtliche Studienende berechnet.
Modulübersicht	Alle Module werden tabellarisch mit Status, Note und ECTS dargestellt. Dadurch sind neben den aggregierten Kennzahlen auch die einzelnen Studienleistungen nachvollziehbar.

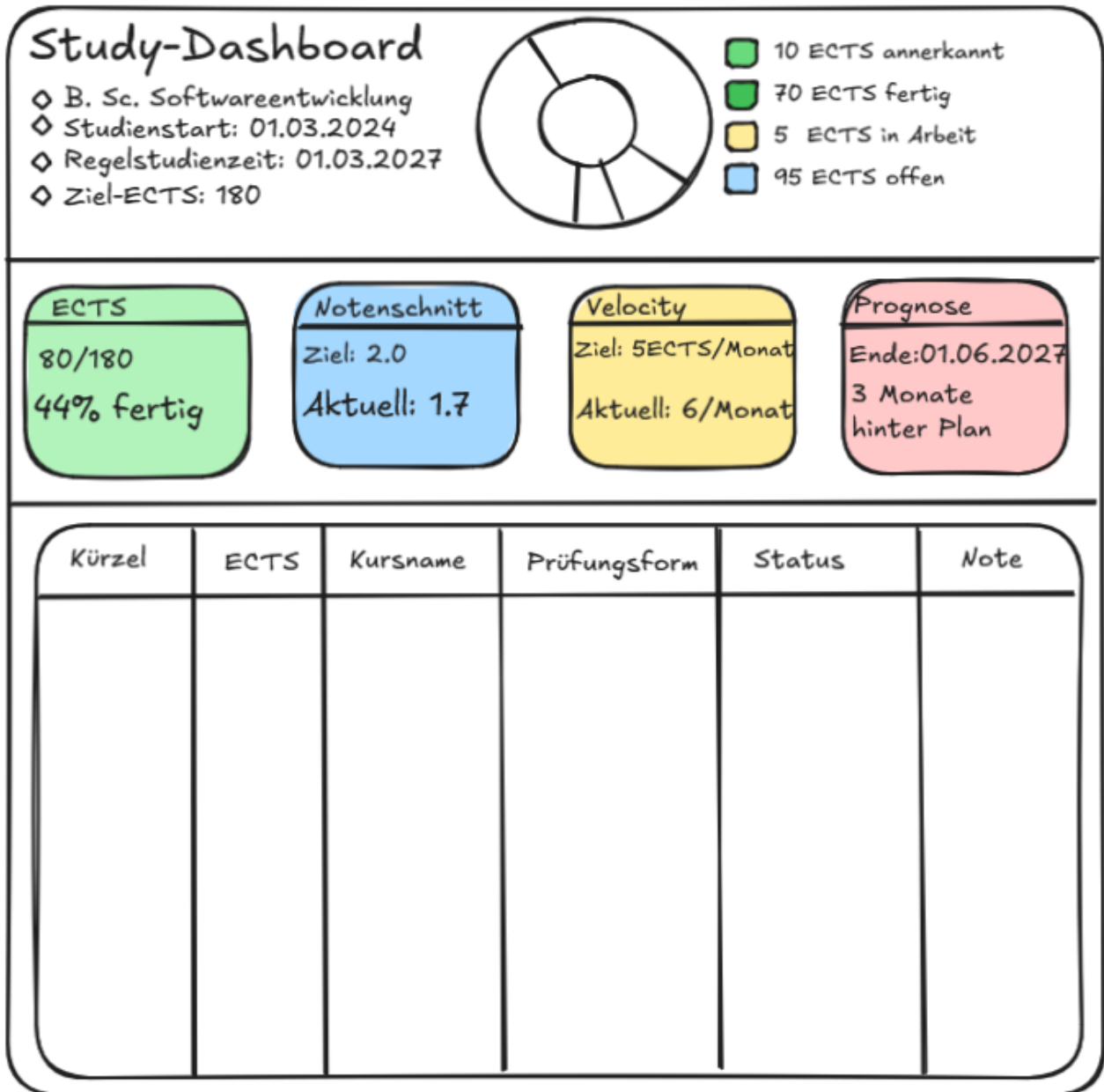
Quelle: Eigene Darstellung.

Die ausgewählten Metriken sind auf das flexible Studieren im IU-Fernstudium ausgerichtet. Da Module unabhängig von einer festen Semesterreihenfolge bearbeitet werden können, steht nicht die Semesterdarstellung im Vordergrund, sondern die kontinuierliche Bewertung des erreichten Fortschritts.

2 Dashboard-Skizze und angezeigte Inhalte

Abbildung 1 zeigt den geplanten Aufbau des Studien-Dashboards. Die Darstellung ist in drei Hauptbereiche gegliedert: einen Kopfbereich mit allgemeinen Studiendaten, einen Kennzahlenbereich mit zentralen Zielgrößen sowie eine detaillierte Modulübersicht.

Abb. 1. Wireframe des geplanten Studien-Dashboards



Quelle: Eigene Darstellung, erstellt mit www.excalidraw.com.

Im oberen Bereich werden der Studiengang, das Startdatum, das Ende der Regelstudienzeit sowie die Ziel-ECTS angezeigt. Zusätzlich visualisiert ein Donutdiagramm die Verteilung der ECTS nach Bearbeitungsstatus. Dabei wird nicht die Anzahl der Module, sondern die Summe der ECTS je Status dargestellt.

Die im Diagramm dargestellten Statuswerte sind *anerkannt*, *fertig*, *in Arbeit* und *offen*. Anerkannte Module werden separat geführt, da sie zwar zum Studienfortschritt beitragen, aber nicht im Rahmen des aktuellen Studiums bearbeitet wurden. Daher sollen sie nicht in die Berechnung der Velocity einfließen. Fertige Module stellen abgeschlossene Leistungen dar. Module mit dem Status *in Arbeit* befinden sich aktuell in Bearbeitung. Offene Module wurden noch nicht begonnen. Das Diagramm gibt somit mit Farblischer Unterstützung einen schnellen Überblick über den Studienfortschritt.

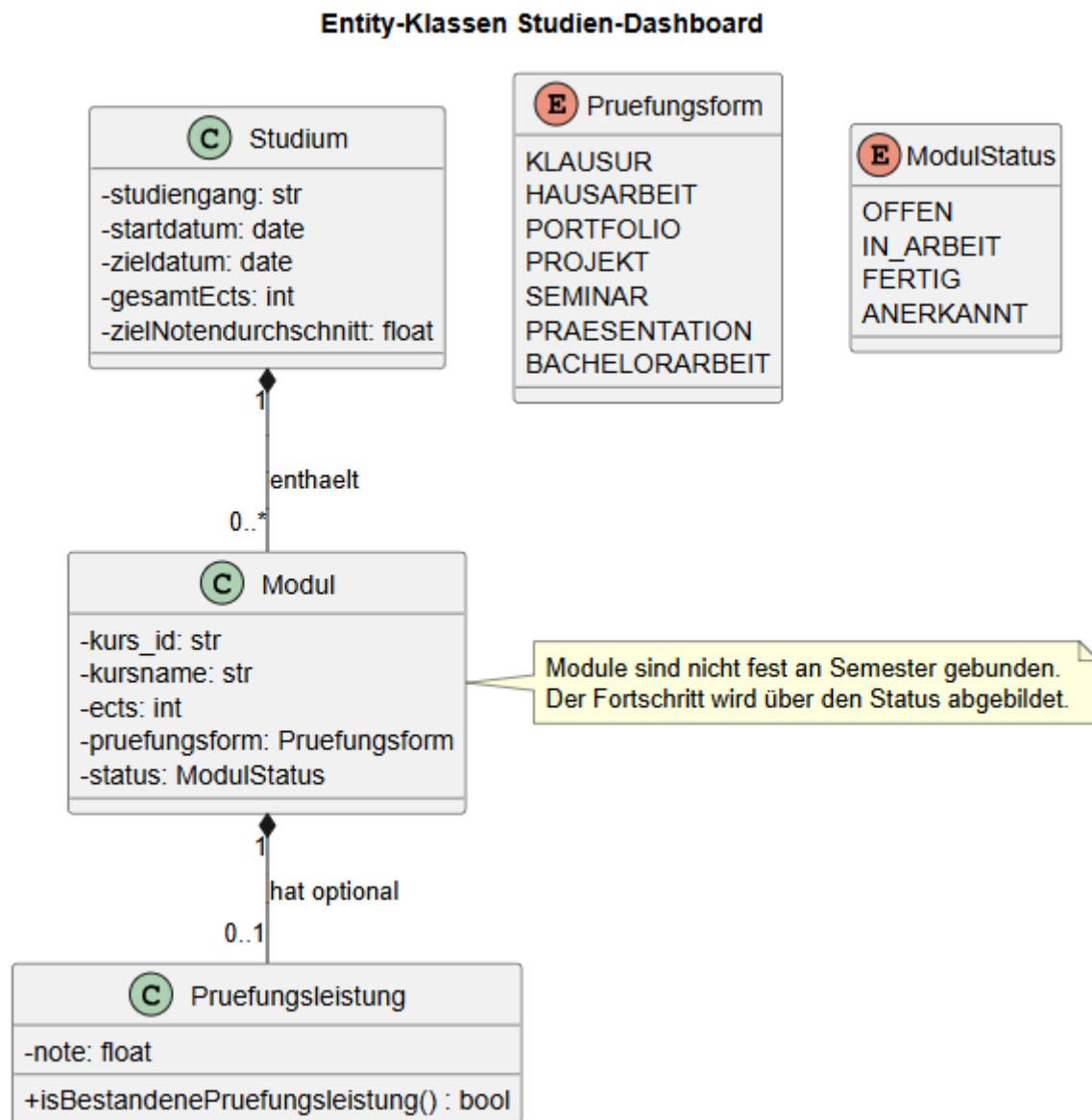
Der mittlere Bereich enthält vier Kennzahlen-Kacheln. Die Kachel *ECTS* zeigt den aktuellen Fortschritt im Verhältnis zu den Ziel-ECTS sowie den prozentualen Anteil der bereits abgeschlossenen und anerkannten ECTS. Die Kachel *Notenschnitt* zeigt den aktuellen gewichteten Durchschnitt und stellt ihn dem Zielwert von 2,0 gegenüber. Die Kachel *Velocity* zeigt die durchschnittlich erreichten ECTS pro Monat durch eigene Leistung seit Studienstart sowie die Ziel-Velocity, die erforderlich ist, um innerhalb der Regelstudienzeit abzuschließen. Die Kachel *Prognose* zeigt das voraussichtliche Studienende auf Basis der bisherigen Velocity und macht sichtbar, ob das Studium voraussichtlich innerhalb der Regelstudienzeit abgeschlossen wird oder zeitlich hinter dem Plan liegt.

Im unteren Bereich befindet sich eine tabellarische Modulübersicht. Sie enthält Kürzel, ECTS, Kursname, Prüfungsform, Status und Note. Dadurch werden die aggregierten Kennzahlen des Dashboards durch Detailinformationen zu den einzelnen Modulen ergänzt. Es wird sichtbar, welche Prüfungsleistungen noch offen sind und welche Module sich aktuell in Bearbeitung befinden. Dies unterstützt insbesondere im Fernstudium die Planung, mit welchem Modul als Nächstes begonnen werden soll. Außerdem eignet sich die tabellarische Darstellung als Grundlage für das spätere Hinzufügen und Bearbeiten von Modulen, beispielsweise zur Ergänzung von Noten oder zur Anpassung von Modulstatus und Stammdaten.

3 Objektorientierte Modellierung

Für die Konzeptionsphase werden nur die fachlichen Entity-Klassen betrachtet, deren Daten später gespeichert werden sollen. Das UML-Klassendiagramm umfasst die Klassen *Studium*, *Modul* und *Prüfungsleistung*. Im Mittelpunkt stehen die speicherbaren Attribute und die Beziehungen zwischen den fachlichen Objekten. Berechnete Dashboard-Kennzahlen wie ECTS-Fortschritt, Velocity, Notenschnitt oder Prognose werden nicht als Attribute gespeichert, sondern später aus den Entity-Daten abgeleitet.

Abb. 2. UML-Klassendiagramm der Entity-Klassen



Quelle: Eigene Darstellung.

Die Klasse *Studium* bildet den fachlichen Rahmen des Dashboards. Sie enthält den Studiengang, das Startdatum, das Zieldatum, die Gesamtzahl der ECTS und den Ziel-Notendurchschnitt. Diese Werte dienen als Vergleichsgrundlage für den tatsächlichen Studienfortschritt.

Die Klasse *Modul* repräsentiert eine fachliche Studieneinheit. Ein Modul besitzt eine Kurs-ID, einen Kursnamen, ECTS, einen Status und eine Prüfungsform. Eine separate Klasse *Semester* wird bewusst nicht modelliert, da Module im IU-Fernstudium flexibel und nicht zwingend in einer festen Semesterreihenfolge bearbeitet werden. Für die zentralen Kennzahlen sind stattdessen Status, ECTS, Note und Studiendauer relevant. Der Fortschritt wird daher primär über den Modulstatus abgebildet.

Status und Prüfungsform werden als Enumerationen modelliert. Der *ModulStatus* ist auf die Werte *offen*, *in Arbeit*, *fertig* und *anerkannt* beschränkt. Die *Prüfungsform* ist auf definierte Werte wie *Klausur*,

Hausarbeit, Portfolio, Projekt, Seminar, Präsentation und *Bachelorarbeit* begrenzt. Dadurch werden ungültige oder uneinheitliche Freitexteingaben vermieden und spätere Auswertungen, zum Beispiel nach Status oder Prüfungsform, erleichtert.

Die Klasse *Prüfungsleistung* enthält prüfungsbezogene Informationen zu einem Modul. In der aktuellen Konzeption wird ausschließlich die Note gespeichert. Der Bestehensstatus wird nicht redundant abgelegt, da er aus der Note abgeleitet werden kann. Die Methode *isBestandenePruefungsleistung()* bildet diese fachliche Regel ab.

Zwischen *Studium* und *Modul* besteht eine Komposition, da die Module im Dashboard als Bestandteile eines konkreten Studienverlaufs betrachtet werden. Die Multiplizität *0..** erlaubt dennoch, dass ein Studium zunächst ohne eingepflegte Module existiert. Zwischen *Modul* und *Prüfungsleistung* wird ebenfalls eine Komposition modelliert, da eine Prüfungsleistung fachlich zu genau einem Modul gehört. Die Multiplizität *0..1* berücksichtigt, dass für offene oder aktuell bearbeitete Module noch keine Prüfungsleistung vorliegen muss.

Auf Vererbung wird bewusst verzichtet, da die betrachteten fachlichen Objekte keine natürliche Generalisierungs oder Spezialisierungsstruktur benötigen. Das Modell bleibt dadurch auf Klassen, Attribute, Enumerationen und Objektbeziehungen fokussiert.

4 Machbarkeitsprüfung und Werkzeugwahl

Zur Vorbereitung wurden mehrere Python-Testprogramme erstellt, um zentrale technische Entscheidungen zu validieren und Umsetzungsrisiken frühzeitig zu minimieren. Die Prüfung erfolgte auf einem Windows-System mit dem Ziel, eine hohe Portabilität und eine einfache Ausführbarkeit durch die Tutorin oder den Tutor sicherzustellen.

Programmiersprache und Entwicklungsumgebung Als Basis wird Python in der Version 3.13.1 eingesetzt, um moderne Sprachfeatures wie Dataclasses und aktuelle Typannotationen nutzen zu können. Die Entwicklung erfolgt in Visual Studio Code. Diese Wahl begründet sich durch die integrierte Unterstützung für das Terminal und die Versionsverwaltung Git, welche für die Finalisierungsphase zwingend gefordert ist. Git ermöglicht zudem eine iterative Entwicklung, bei der Konzeption, Reflexion und Prototyping durch Branches oder Tags sauber voneinander getrennt bleiben.

Verzicht auf externe Bibliotheken Es wird bewusst auf externe Pakete verzichtet. Durch die ausschließliche Nutzung der Python-Standardbibliothek bleibt die Anwendung vollständig offline lauffähig und unabhängig von Paketquellen. Dies garantiert, dass der Prototyp mit geringem Aufwand und ohne zusätzliche Installationen über *pip* auf dem Prüfungssystem gestartet werden kann.

Datenspeicherung und Persistenz Die Speicherung der fachlichen Entity-Daten erfolgt in CSV-Dateien. Diese Wahl ist für die geringe Datenmenge des Prototyps geeignet, da CSV-Dateien transparent sind und ohne Datenbank-Server auskommen. Ein Testprogramm bestätigte die Machbarkeit: Moduldaten wurden persistent gespeichert, eingelesen und über eine ID erfolgreich gelöscht. Mit der Standardbibliothek CSV ist das schreiben von CSV-Dateien sehr einfach erreichbar siehe Abbildung 3

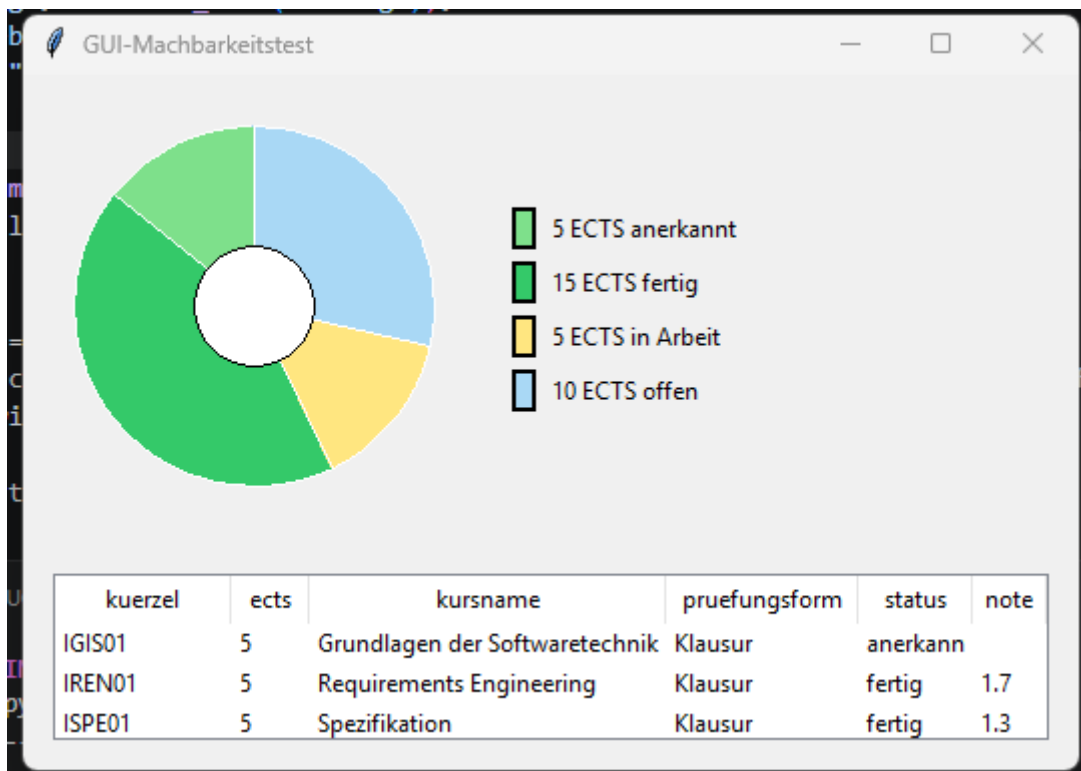
Abb. 3. CSV-Datei schreiben

```
1 def createModul(self, id, titel):
2     with open(self.dateiname, "a", newline="", encoding="utf-8") as datei:
3         writer = csv.DictWriter(datei, fieldnames=self.header)
4         writer.writerow({"id": id, "titel": titel})
```

Quelle: Eigene Darstellung.

Benutzerschnittstelle (Tkinter vs. CLI) Zunächst wurde eine reine Kommandozeilenlösung (CLI) evaluiert. Diese wäre für einfache Eingaben und Ausgaben ausreichend, hätte jedoch die geplante schnelle Erkennbarkeit der Zielerreichung durch Diagramme und Kennzahlen-Kacheln stark eingeschränkt. Daher fiel die Wahl auf Tkinter. Da Tkinter Teil der Python-Standardbibliothek ist, ermöglicht es eine grafische Oberfläche ohne externe Abhängigkeiten. In einer Machbarkeitsstudie wurde erfolgreich geprüft, ob zentrale Elemente wie ein Donutdiagramm zur ECTS-Verteilung über das Canvas-Widget und tabellarische Ansichten umsetzbar sind. Das Ergebnis des GUI-Tests wird durch einen Screenshot der erzeugten Tkinter-Oberfläche belegt siehe Abb. 4.

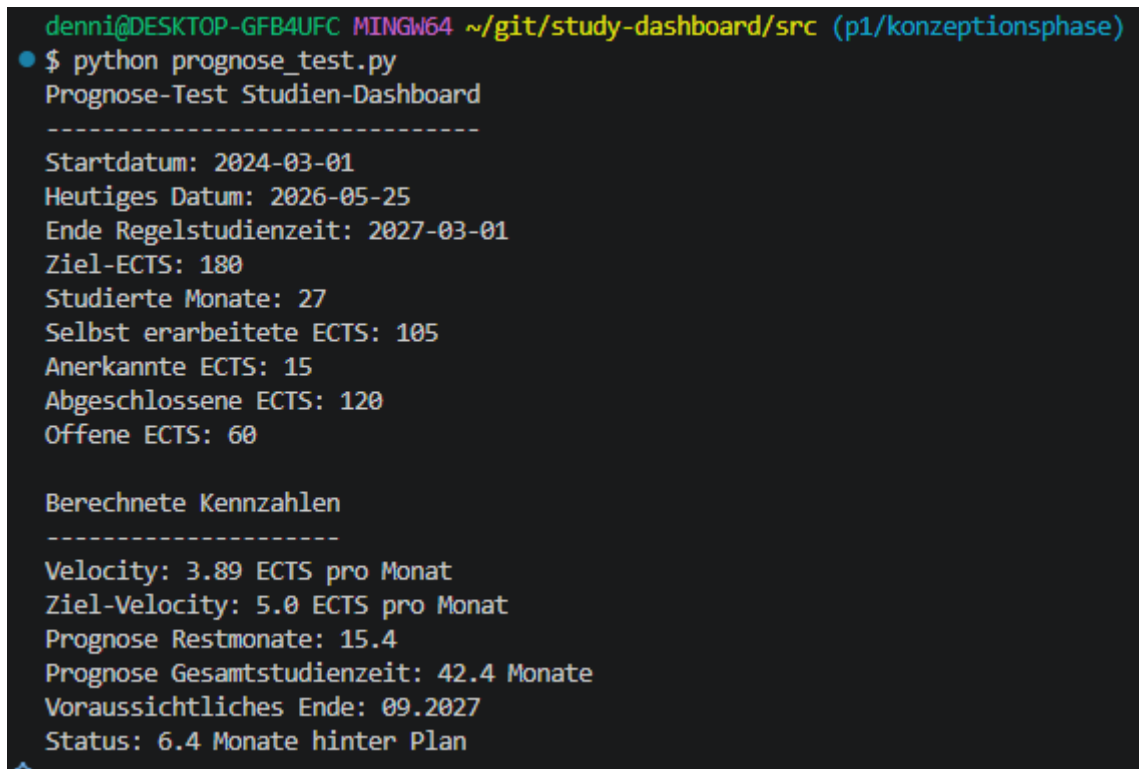
Abb. 4. GUI-Machbarkeitstest mit Tkinter



Quelle: Eigene Darstellung.

Berechnungslogik und Prognose-Metriken Ein spezielles Testprogramm validierte die mathematische Ableitung der Velocity, also der ECTS pro Monat, und der voraussichtlichen Studienzeit aus den Parametern Startdatum und Ziel-ECTS. Dies zeigt, dass die geplanten Kennzahlen dynamisch aus dem Domain-Modell ohne zusätzliche Bibliotheken berechnet werden können. Die erfolgreiche Ausführung der Berechnungen wird zusätzlich durch einen Screenshot der Konsolenausgabe dokumentiert Abb. 5.

Abb. 5. Konsolenausgabe des CSV- und Prognose-Machbarkeitstests



```
denni@DESKTOP-GFB4UFC MINGW64 ~/git/study-dashboard/src (p1/konzeptionsphase)
$ python prognose_test.py
Prognose-Test Studien-Dashboard
-----
Startdatum: 2024-03-01
Heutiges Datum: 2026-05-25
Ende Regelstudienzeit: 2027-03-01
Ziel-ECTS: 180
Studierte Monate: 27
Selbst erarbeitete ECTS: 105
Anerkannte ECTS: 15
Abgeschlossene ECTS: 120
Offene ECTS: 60

Berechnete Kennzahlen
-----
Velocity: 3.89 ECTS pro Monat
Ziel-Velocity: 5.0 ECTS pro Monat
Prognose Restmonate: 15.4
Prognose Gesamtstudienzeit: 42.4 Monate
Voraussichtliches Ende: 09.2027
Status: 6.4 Monate hinter Plan
```

Quelle: Eigene Darstellung.

Fazit der Machbarkeit Die durchgeführten Tests bestätigen, dass die Kombination aus CSV-Persistenz, Tkinter-GUI und mathematischer Prognoseberechnung mit Python 3.13.1 und der Standardbibliothek prototypisch vollständig umsetzbar ist.